

Pythonfördjupning: Aktieanalysverktyg med maskininlärning

Analys, prediktion och beslutsstöd i datadriven miljö



Katarina Persson
EC-Utbildning
Pythonfördjupning
2026-06-28

Inledning

Aktiemarknaden uppfattas som transparent men präglas i praktiken av informationsasymmetrier, varierande datakvalitet och analysmetoder som ofta bygger på manuella moment och subjektiva bedömningar. Detta gör det svårt att genomföra konsekventa och reproducerbara analyser.

I stället för att försöka förutsäga exakta priser fokuserar projektet på riktning — uppgång, nedgång eller neutral rörelse — eftersom riktning är mindre brusigt och mer stabilt för tabulär finansiell data. En sådan modell fungerar som ett beslutsstöd som levererar sannolikheter, inte köp- eller säljsignaler.

Att förutsäga finansiella marknadsrörelser är metodologiskt komplext. Marknader präglas av hög volatilitet, lågt signal-till-brus-ratio och externa faktorer som gör att modeller ofta presterar nära slumpnivå. Under projektets gång blev det tydligt att den största utmaningen inte var modellvalet, utan att definiera riktning på ett statistiskt och finansiellt robust sätt. Target-design visade sig vara en av de mest avgörande och svårhanterliga komponenterna i finans-ML, särskilt i en volatil och icke-stationär tidsserie där signal och brus är svåra att separera.

Problemformulering

Verktyg för aktieanalys är ofta spridda över flera plattformar och saknar enhetliga metoder för att kombinera tekniska indikatorer med maskininlärning. Detta skapar ett behov av en strukturerad modell som kan integrera relevanta marknadsfaktorer i ett reproducerbart analysflöde.

Syfte

Syftet med projektet är att utveckla ett datadrivet ramverk som kombinerar tekniska indikatorer och maskininlärning för att analysera OMXS30-bolag. Genom att studera historiska prisrörelser ska modellen identifiera vilka variabler som har störst signalstyrka och därmed kan bidra till en metodologiskt neutral riktningsbedömning.

Metod

Projektfrågorna i aktieprogrammet transformerades till ett maskininlärningsproblem formulerat som en multiklass-klassificering med tre klasser: uppgång, nedgång och neutral rörelse. Detta minskar brus vid små prisförändringar och ger en stabilare bedömning av riktning än exakta prisprognoser.

I praktiken visade det sig att frågan inte var *om* riktning kan förutsägas, utan *hur* riktning ska definieras. Prisdata innehåller både brus och signal, och rörelser varierar mellan olika volatilitetsregimer. För att konstruera en meningsfull target-variabel måste man därför avgöra vad som utgör en faktisk rörelse, vad som är neutral zon, hur mycket prisförändring som krävs för att klassas som positiv eller negativ, och hur dessa gränser ska anpassas efter volatilitet. Detta blev en central del av projektet och en avgörande komponent för modellernas prestanda.

Data Engineering

Datahanteringen bygger på en strukturerad ETL-pipeline där rådata från OMXS30 bearbetas till ett konsistent och modellvänligt dataset. Processen omfattar laddning av prisdata, beräkning av tekniska indikatorer och konstruktion av ett rikt feature-set. Samtliga steg är implementerade i separata builder-klasser för att säkerställa modularitet, reproducerbarhet och tydlig separation av ansvar.

Datan delas upp i fyra volatilitetsgrupper (G1–G4) baserat på historisk prISRörelse. Detta minskar heterogenitet och gör det möjligt att träna modeller som är bättre anpassade till olika marknadsregimer. Varje grupp får ett eget dataset med samma feature-struktur men olika statistiska egenskaper.

Pipeline-logiken är helt tidsserieanpassad: alla transformationer sker kronologiskt, och train/val/test-perioder hålls strikt separerade för att undvika dataläckage. Resultatet är ett stabilt och konsekvent dataset som kan användas för modellträning, optimering och sluttestning.

OOP-strukturen

Projektet använder en enkel objektorienterad struktur för att organisera dataladdning, feature-generering och modellträning. Varje huvudsteg i pipeline-logiken kapslas i separata klasser, vilket ger tydlig separation av ansvar och gör koden modulär och lätt att utöka. Klassen för dataladdning hanterar prisdata och tekniska indikatorer, medan feature-buildern skapar det fullständiga feature-setet som används av modellerna.

Modellklasserna ansvarar för träning, optimering och retraining per volatilitetsgrupp (G1–G4). Genom att skapa en instans per grupp kan samma metodik återanvändas konsekvent, samtidigt som gruppsspecifika parametrar och resultat hålls isolerade. Denna struktur gör pipeline-flödet reproducerbart och minskar risken för dataläckage eller inkonsistens mellan grupperna.

OOP-upplägget är avsiktligt enkelt och fokuserar på att organisera kod och dataflöde snarare än att bygga ett komplext system. Det ger en ren struktur där databehandling, modellering och utvärdering hålls åtskilda, vilket underlättar både felsökning och vidareutveckling.

Datainsamling och rådatahantering

Datasetet bygger på OMXS30-bolagen, vars höga likviditet ger stabil och relativt brusfri prisdata som lämpar sig för tekniska indikatorer. Historiska OHLCV-värden hämtas via yfinance, vilket säkerställer ett konsekvent och reproducerbart datamaterial för feature-bearbetning och modellering.

Projektet använder fem års historiska data för OMXS30. Trots att kortsiktiga modeller ofta baseras på 6–12 månaders historik valdes en längre period för att fånga flera marknadscyklar. Fem år ger tillräcklig variation i volatilitet och trend för att modellen ska kunna identifiera robusta mönster över olika marknadsregimer, samtidigt som perioden fortfarande speglar dagens marknadsstruktur.

All feature-konstruktion utgår från dessa historiska kurser, där tekniska indikatorer och prisbaserade nyckeltal beräknas direkt från rådatan. Efter extraktionen genomgår datan validering och harmonisering för att hantera saknade värden och säkerställa ett stabilt och enhetligt flöde genom pipeline.

Feature-byggnad i builder-klasserna

Feature-konstruktionen baseras på historiska prisserier och genereras av separata builder-moduler. TechBuilder beräknar tekniska indikatorer, BandBuilder skapar band- och volatilitetsmått och PriceCombinatoricBuilder genererar prisbaserade kombinationer och nyckeltal. Varje builder kapslar in sin logik och producerar funktioner i ett standardiserat format som kan användas direkt i modelleringen.

Samtliga builders följer samma strukturella mönster med separerade interna steg och ett enhetligt gränssnitt via `build_one()` och `build_all()`. Detta ger en modulär och reproducerbar feature-pipeline där nya feature-typer kan läggas till utan att påverka befintliga komponenter. Designen säkerställer tydlig separation av ansvar och gör feature-lagret skalbart och lätt att vidareutveckla.

BandBuilder och Target-design som central utmaning

Under projektets gång blev det tydligt att den centrala utmaningen inte var modellvalet, utan att definiera riktning i prisrörelser på ett statistiskt och finansiellt robust sätt. Prisdata är småskaliga, brusiga och volatilitetsberoende, vilket gör det svårt att avgöra vad som ska klassas som uppgång, nedgång eller neutral rörelse. Detta gjorde band-design och target-konstruktion till en avgörande del av projektet.

Target-variabeln konstrueras genom att klassificera framtida prisrörelser i tre kategorier: uppgång, nedgång och neutral rörelse. Klassgränserna baseras på historiska percentiler inom respektive volatilitetsgrupp, vilket gör target-designen anpassad till marknadsregimen. Detta minskar brus och ger en mer stabil klassfördelning än fasta trösklar.

Target-beräkningen sker helt kronologiskt för att undvika dataläckage. Varje grupp får sina egna bandgränser och sin egen target-fördelning, vilket gör att modellerna tränas på targets som är statistiskt rimliga för just den gruppens volatilitet. Denna design är central för att modellerna ska kunna lära riktning på ett robust sätt.

Band-design är en välkänd utmaning inom finans-ML eftersom marknadsdata är skev, fat-tailed och heteroskedastisk. Percentiler beter sig annorlunda än i normalfördelade data, mellanzonen tenderar att bli instabil och volatiliteten varierar över tid och mellan olika tickers. BandBuilder delar upp datasetet i fyra volatilitetsgrupper (G1–G4) baserat på historiska prisrörelser. Syftet är att minska heterogenitet och skapa mer enhetliga marknadsregimer där modellerna kan lära tydligare mönster. Varje grupp får ett eget dataset med samma feature-struktur men olika statistiska egenskaper, vilket gör det möjligt att optimera modellerna separat per volatilitetsnivå.

Tidsseriehantering och dataläckage

Eftersom datasetet består av historiska prisserier hanteras all data som en tidsserie där ordningen är strikt bevarad. Samtliga builder-moduler arbetar sekventiellt och genererar features utan att använda framtida värden. Train-, validerings- och testdelar delas upp kronologiskt för att undvika att information från senare perioder påverkar tidigare steg.

Tekniska indikatorer och prisbaserade nyckeltal beräknas med omsorg för att undvika dataläckage. Alla beräkningar använder endast data som är tillgänglig fram till respektive tidssteg, och samtliga feature-moduler är konstruerade för att inte introducera framtidsinformation. Denna design säkerställer att både feature-lagret och det slutliga datasetet är reproducerbart och korrekt för tidsserie-modellering.

Modellering

Tre modeller används i projektet: RandomForest, HistogramGradientBoosting och CatBoost. Dessa valdes för att representera både bagging- och boosting-metoder och därmed ge en robust uppsättning algoritmer för brusiga marknadsdata. Datasetet delas upp i fyra volatilitetsgrupper (G1–G4), och samtliga modeller tränas med en strikt tidsserie-split för att undvika dataläckage.

Modelleringen genomförs i tre steg. Först tränas varje modell med standardparametrar för att etablera en baslinje per grupp. Därefter optimeras hyperparametrarna med Optuna, där varje modell optimeras separat inom respektive grupp för att ta hänsyn till skillnader i volatilitet och signalstyrka. Slutligen retränas modellerna på train+val-datan med de optimerade parametrarna och utvärderas på ett helt orört testset.

Modellerna utvärderades på en valideringsperiod som representerar framtida data inom respektive grupp (G1–G4). Resultaten visar tydliga skillnader mellan modellerna och mellan grupperna, vilket speglar variationer i volatilitet och trendstyrka.

Resultat

Samtliga modeller tränades på samma feature-set (85 variabler) och med tidsserieanpassad train/val/test-split. Vid den initiala valideringen visade gradientboosting-metoderna (HGB och CatBoost) konsekvent högst prestanda i alla grupper (G1–G4), medan RF fungerade som en stabil baseline men med lägre träffsäkerhet, särskilt i grupper med hög volatilitet. Valideringsaccuracy låg i intervallet 0.94–0.97 för HGB och CatBoost, medan RF låg mellan 0.70–0.77 beroende på grupp.

Hyperparameteroptimeringen med Optuna gav tydliga förbättringar i samtliga modeller. RF förbättrades kraftigt i G2–G3 och nådde efter optimering nivåer runt 0.95–0.96, medan HGB och CatBoost redan låg nära sina toppvärden och finjusterades ytterligare. Optimeringen bekräftade att HGB är den mest stabila modellen över grupperna, medan CatBoost är särskilt stark i grupper med tydligare trendmönster. Dessa optimerade parametrar låg till grund för retraining-steget.

Efter retraining på train+val-datan ökade prestandan ytterligare. Sluttestningen på det helt orörda testsetet visar att alla tre basmodeller presterar högt i samtliga grupper, med accuracy-värden i intervallet **0.95–0.98**. Resultaten är konsekventa över volatilitetsregimerna, vilket tyder på att segmenteringen i G1–G4 har minskat heterogenitet och gjort det möjligt för modellerna att lära mer enhetliga mönster.

Ensemblemodellen, som kombinerar sannolikheterna från de tre retränade modellerna, uppvisar genomgående den högsta prestandan. I samtliga grupper når ensemblen

accuracy-värden mellan 0,969 och 0,979, vilket är högre än eller i nivå med de bästa enskilda modellerna. Detta visar att ensemblemetoden effektivt fångar komplementära styrkor hos basmodellerna och ger en mer robust och stabil prediktion.

Sammantaget visar resultaten att pipeline-upplägget — med segmentering, optimering, retraining och ensemble — ger en konsekvent hög prediktionsförmåga över alla volatilitetsmiljöer. Modellerna är stabila, förbättras av optimeringen och presterar starkt även på osedd data, vilket indikerar att den valda metodiken är välfungerande för den aktuella problemställningen.

Testning av Data Engineering-pipelinen

Testningen av data-pipen genomfördes i flera lager för att säkerställa korrekthet och reproducerbarhet. Enhetstester verifierade att varje builder-klass hanterade tekniska indikatorer, saknade värden och edge cases korrekt. Pipeline-tester kontrollerade att hela flödet — från rådata till färdigt dataset — fungerade som helhet och att tidsserieordningen var intakt utan dataläckage. Property-based testing användes för att identifiera oväntade indata och numeriska edge cases, medan statisk kodanalys (pylint, mypy, ruff) säkerställde kodkvalitet och typkorrekthet. Slutligen genomfördes sanity-tester av datasetet för att verifiera datatyper, saknade värden och target-fördelning innan modelleringen.